

CSE 105: Data Structures and Algorithms-I (Part 2)

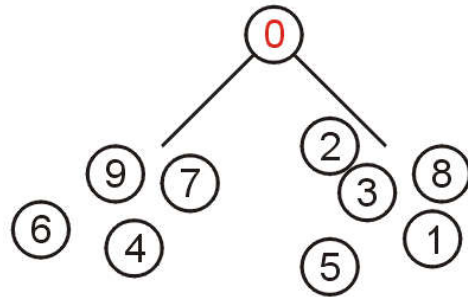
Instructor
Dr Md Monirul Islam

Heap, Heapsort and Priority Queue

Definition: Heap

A non-empty binary tree is a heap if

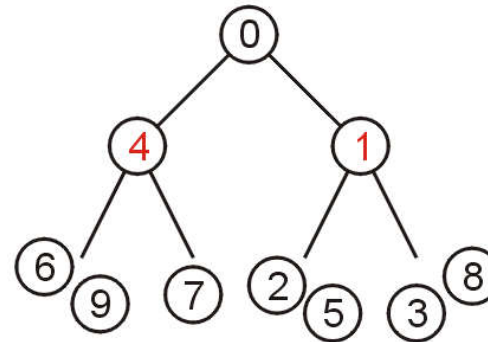
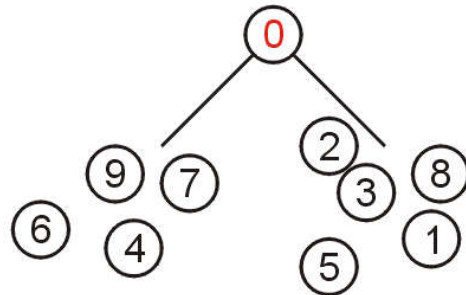
- The key associated with the root maintains certain property with the keys associated with either of the sub-trees (if any)



Definition: min-Heap

A non-empty binary tree is a **min-heap** if

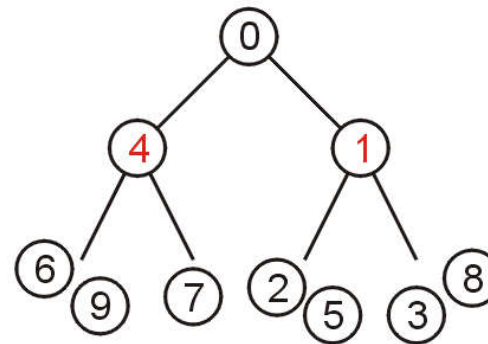
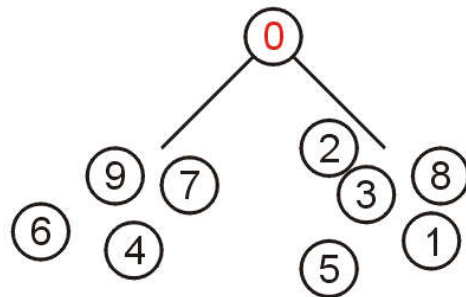
- The **key** associated with the **root** is **less than or equal** to the **keys** associated with either of the **sub-trees** (if any)



Definition: min-Heap

A non-empty binary tree is a **min-heap** if

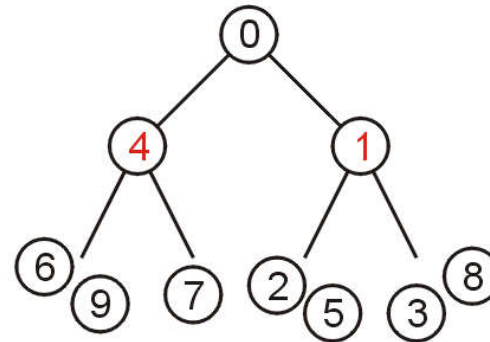
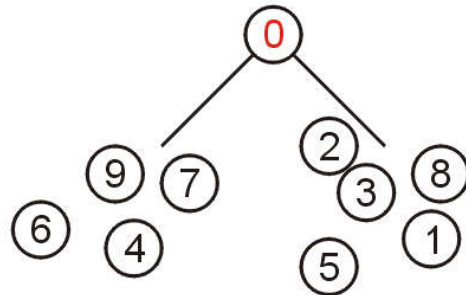
- The **key** associated with the **root** is **less than or equal** to the **keys** associated with either of the **sub-trees** (if any)
- **Both** of the **sub-trees** (if any) are also **binary min-heaps**



Definition: min-Heap

A non-empty binary tree is a **min-heap** if

- The **key** associated with the **root** is **less than or equal** to the **keys** associated with either of the **sub-trees** (if any)
- **Both** of the **sub-trees** (if any) are also **binary min-heaps**



From this definition:

- A **single node** is a **min-heap**
- **All keys** in either sub-tree are **greater** than the **root key**

Definition

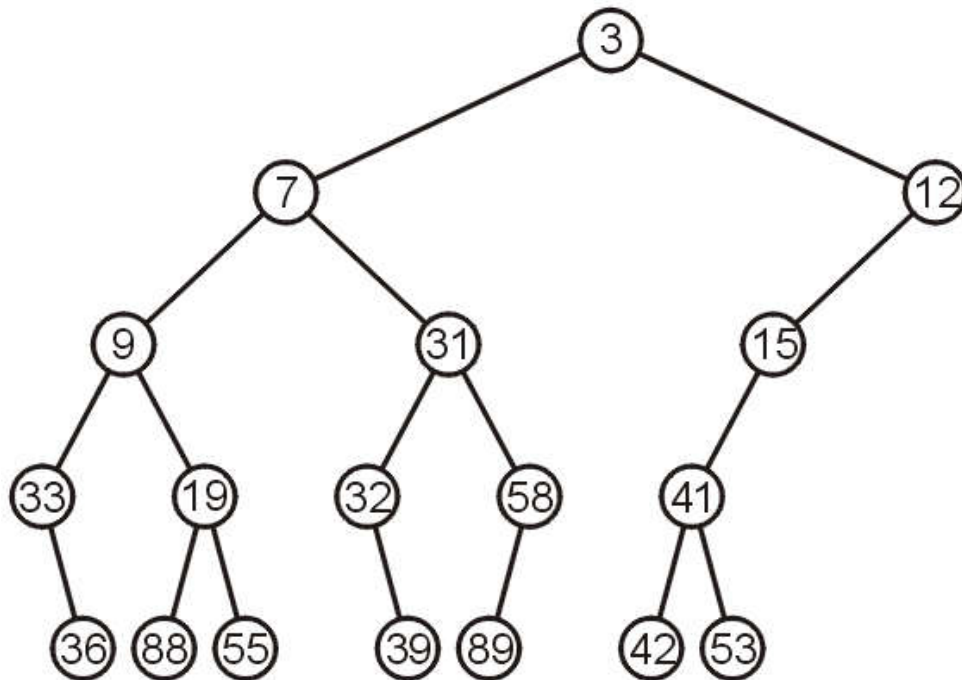
Important:

**There is NO other RELATIONSHIP between
the elements in the TWO SUBTREES
[unlike the BST]**

DON'T fail to understand this!

Example

This is a **binary min-heap**:

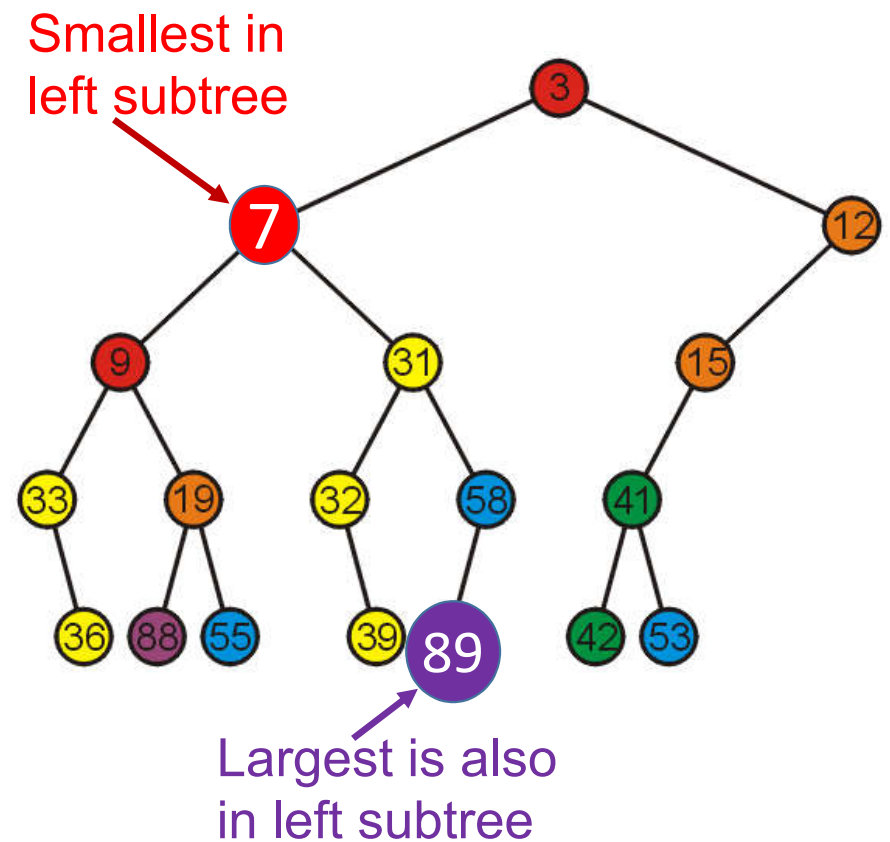


Note: Later, we will implement heap only with complete binary tree...

Example

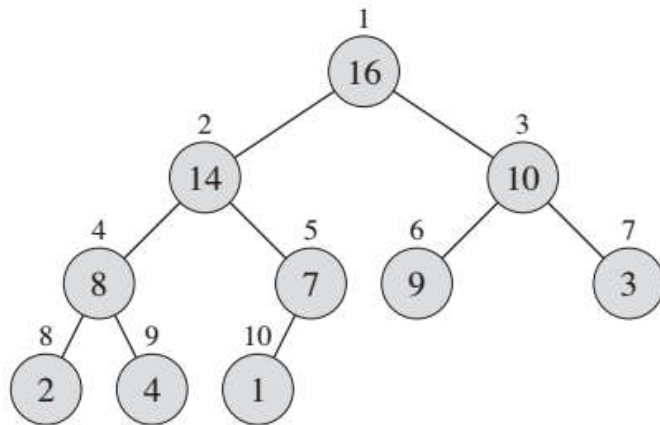
Adding color, we observe

- The left subtree has the **smallest** (7) and the **largest** (89) objects
- No relationship between items with similar priority
 - (just assume for now that the keys are priority values)



Example

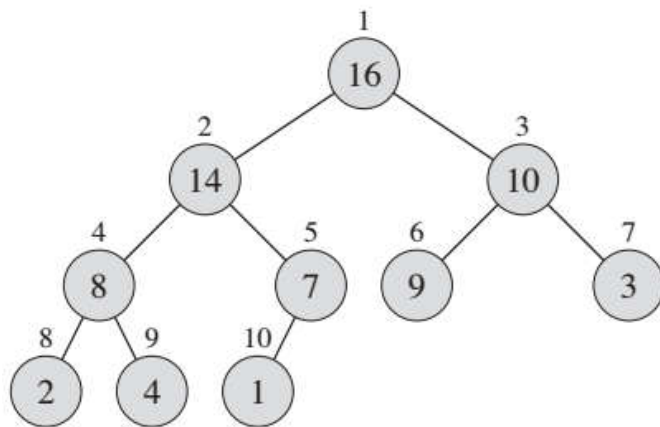
This is a **binary max-heap**:



Note: This max-heap is implemented with complete binary tree...

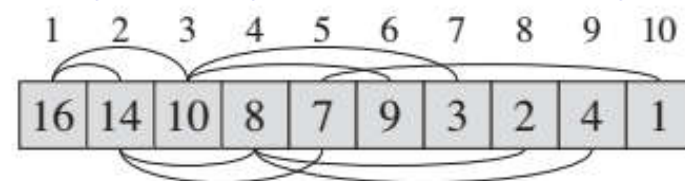
Example

This is a **binary max-heap**:



Note: This max-heap is implemented with complete binary tree...

Easy to represent in array



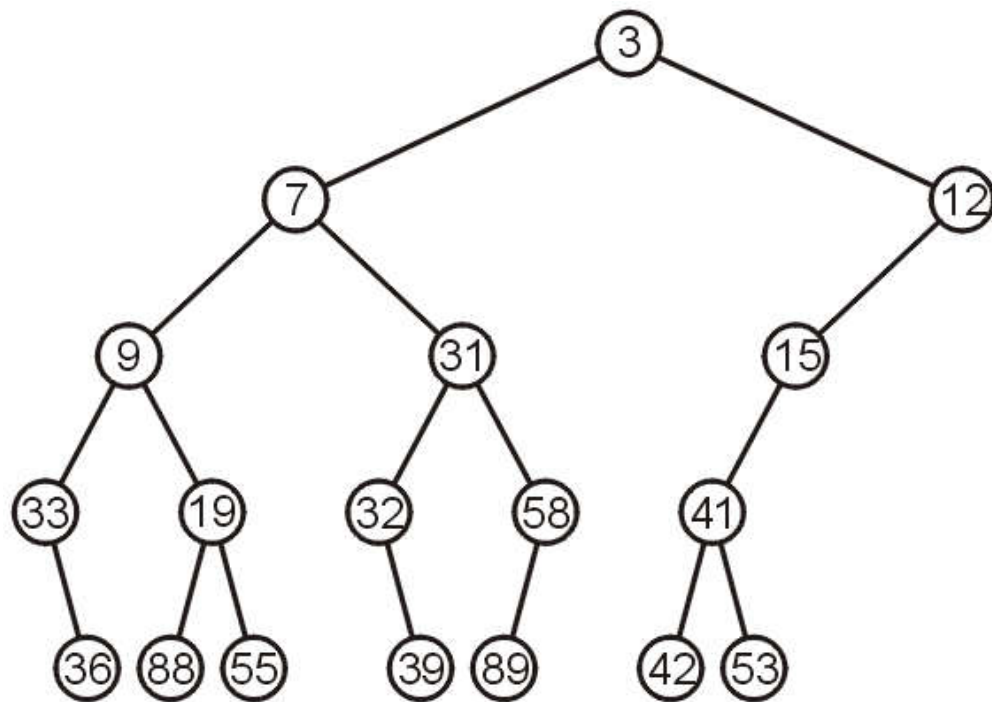
Operations

We will consider three operations:

- Top [actually, `getMinimum` in this case]
- Pop [`remove` or `extractMinimum`]
- Push [`insert`]

Example

We can find the top (or *minimum*) object in $\Theta(1)$ time: 3



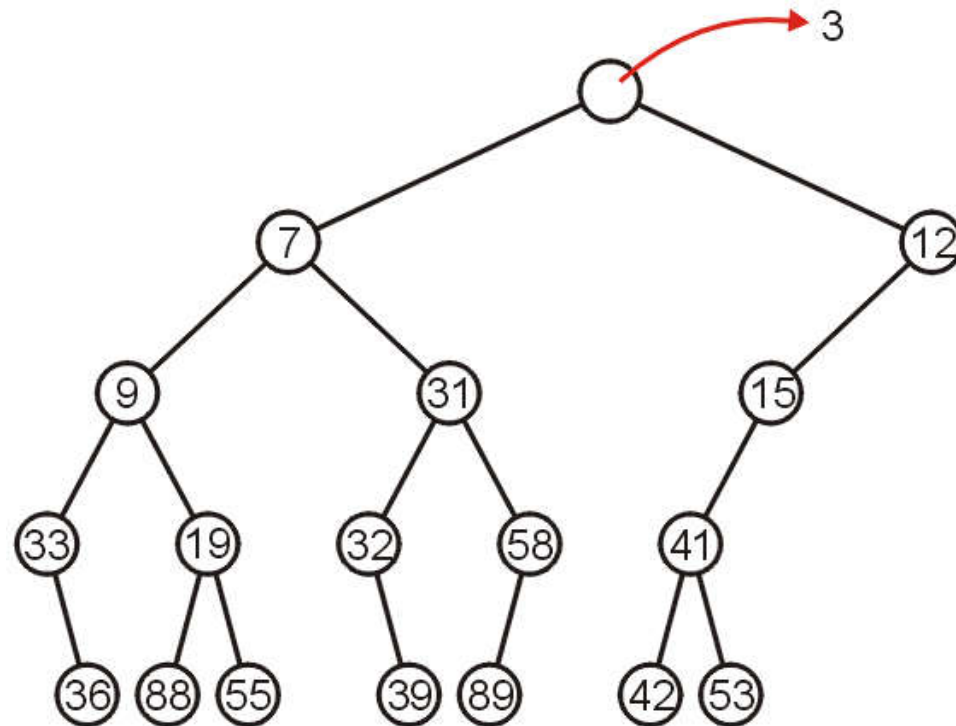
Pop

To **remove the minimum** object:

- Promote the node of the sub-tree which has the least value
- Recurs down the sub-tree from which we promoted the least value

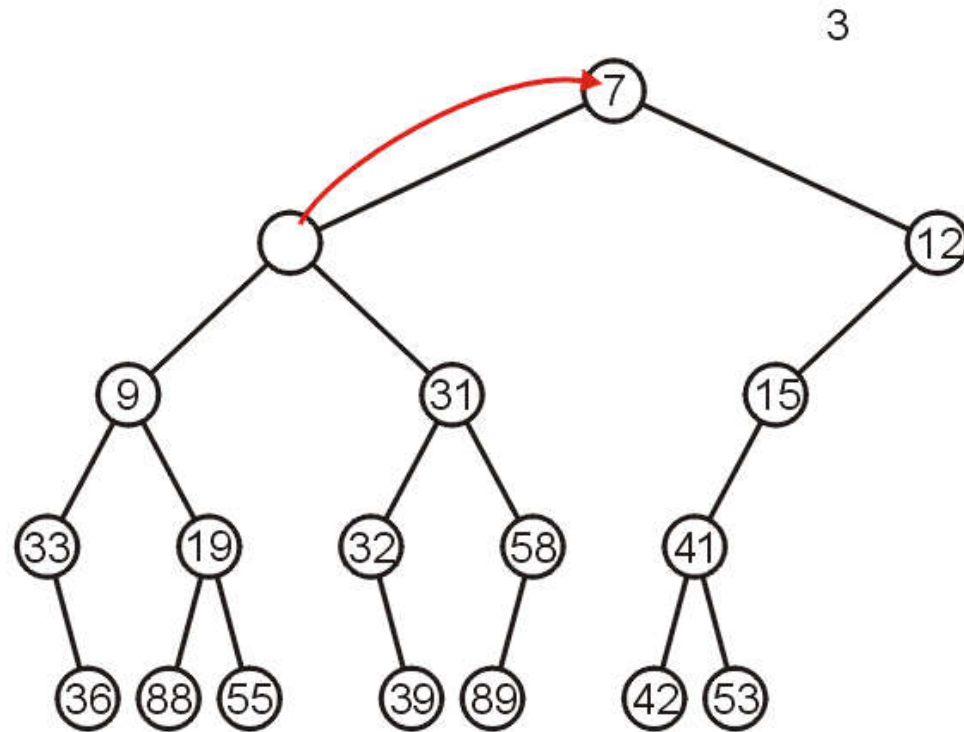
Pop

Using our example, we remove 3:



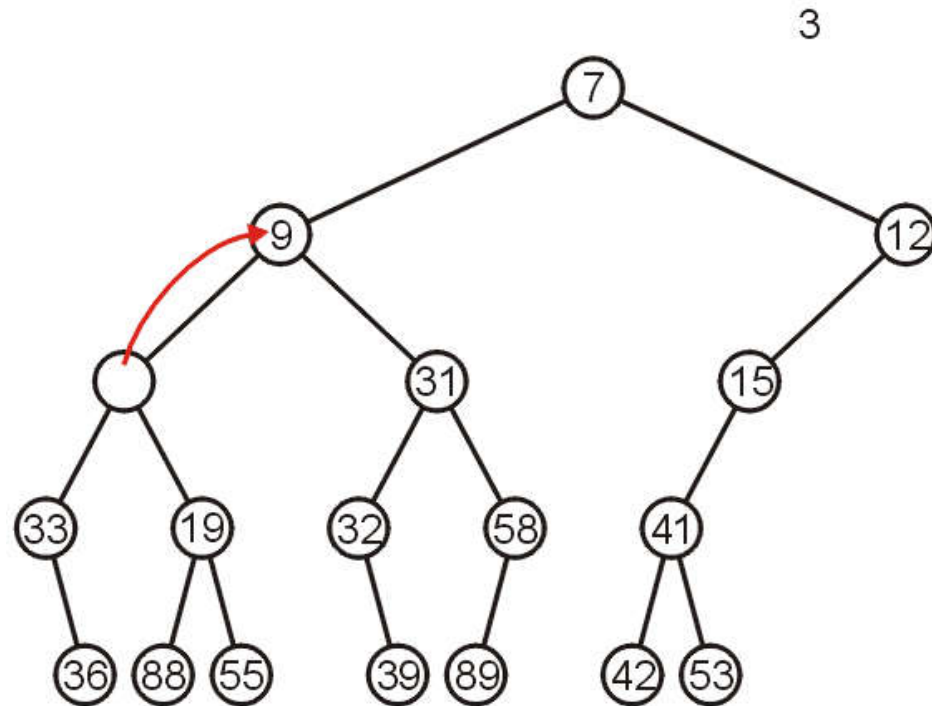
Pop

We promote 7 (the minimum of 7 and 12) to the root:



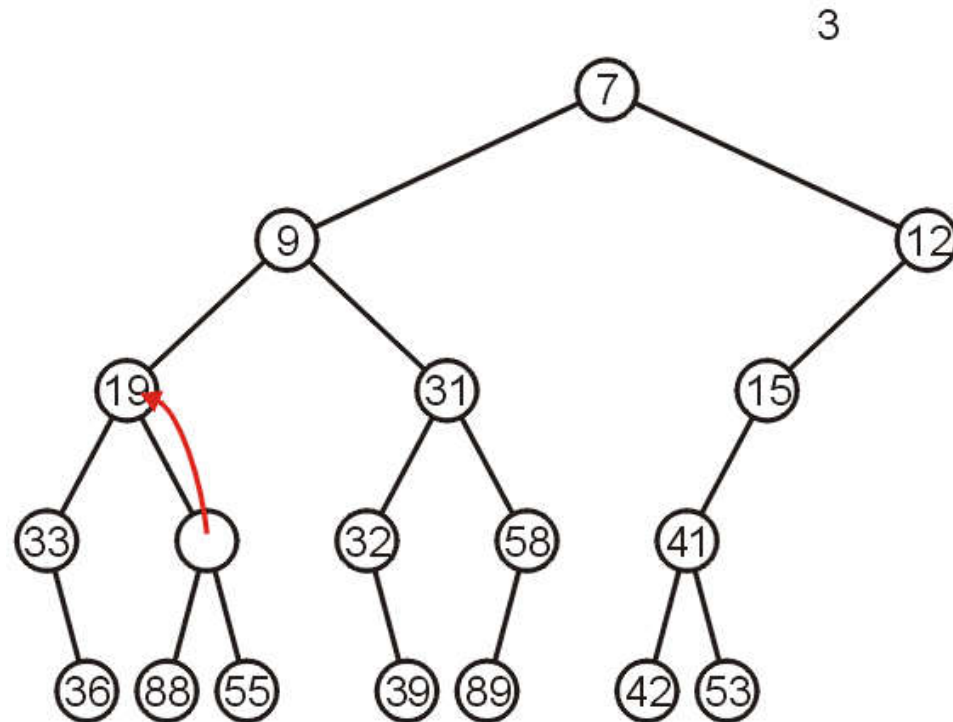
Pop

In the left sub-tree, we promote 9:



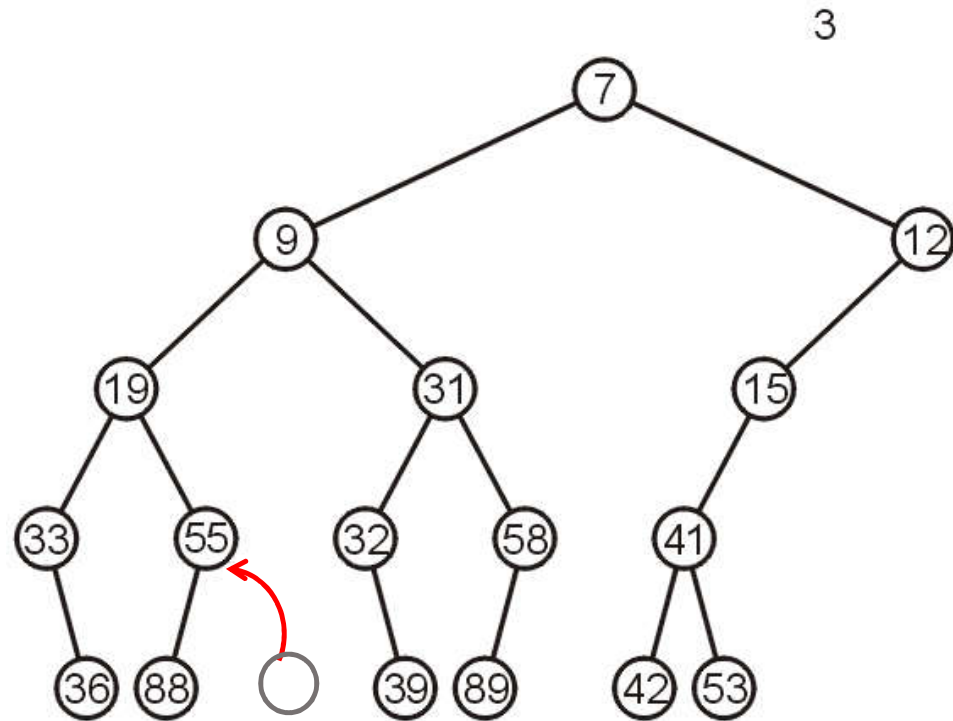
Pop

Recursively, we promote 19:



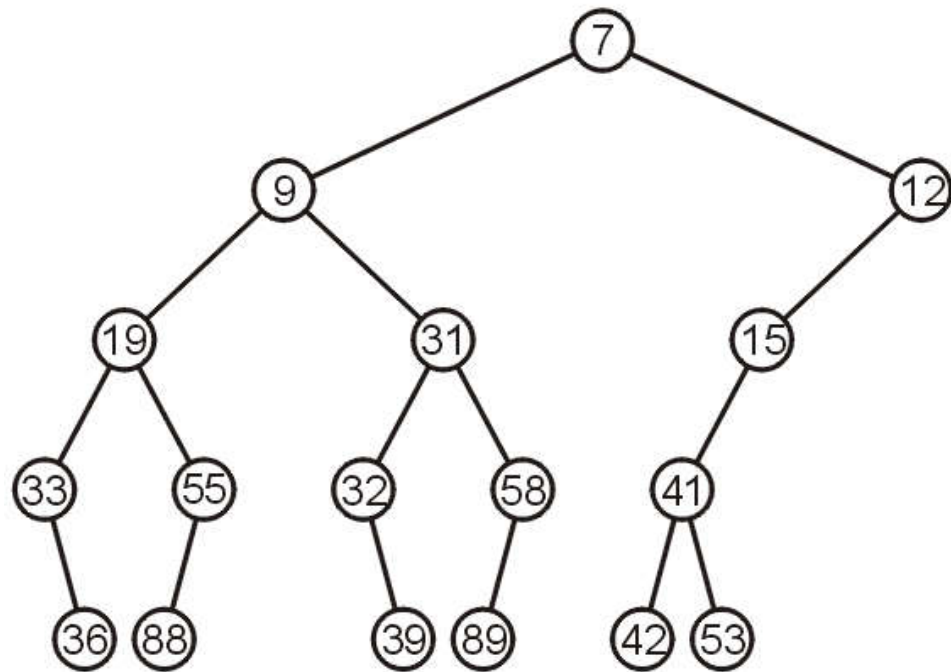
Pop

Finally, 55 is a leaf node, so we promote it and delete the leaf



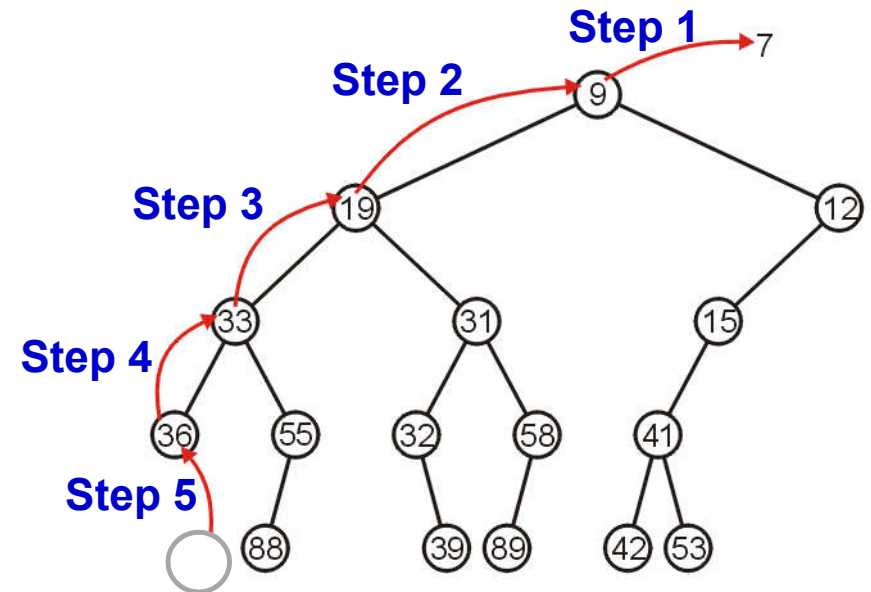
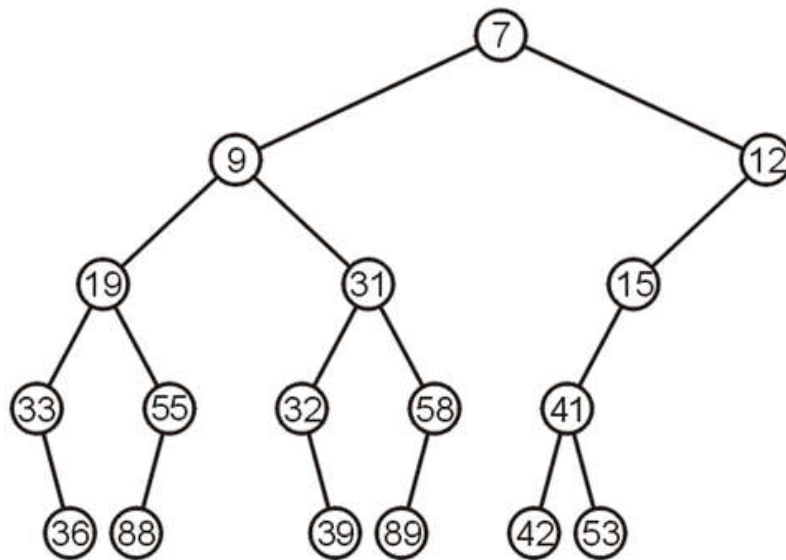
Pop

This is **now** a **min-heap**



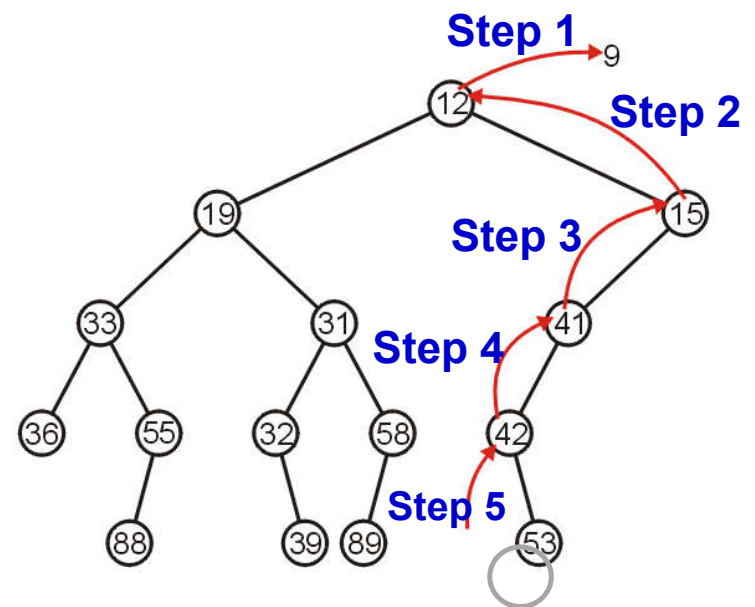
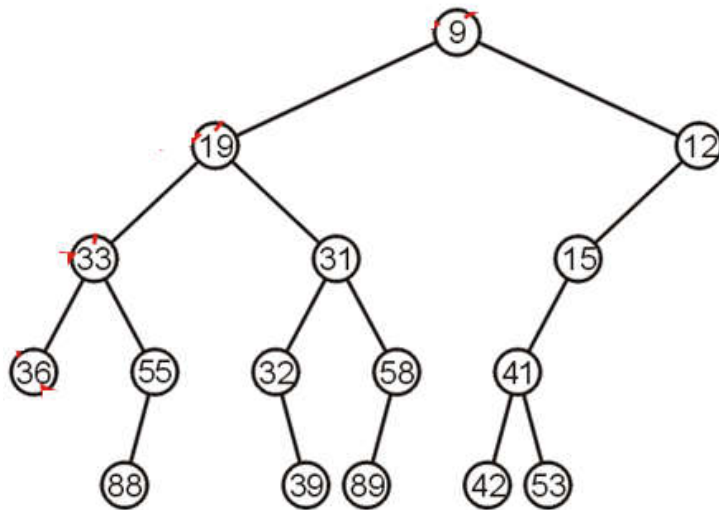
Pop

Repeating this operation again, we can remove 7:



Pop

If we remove 9, we **must now promote from the right sub-tree:**



Push

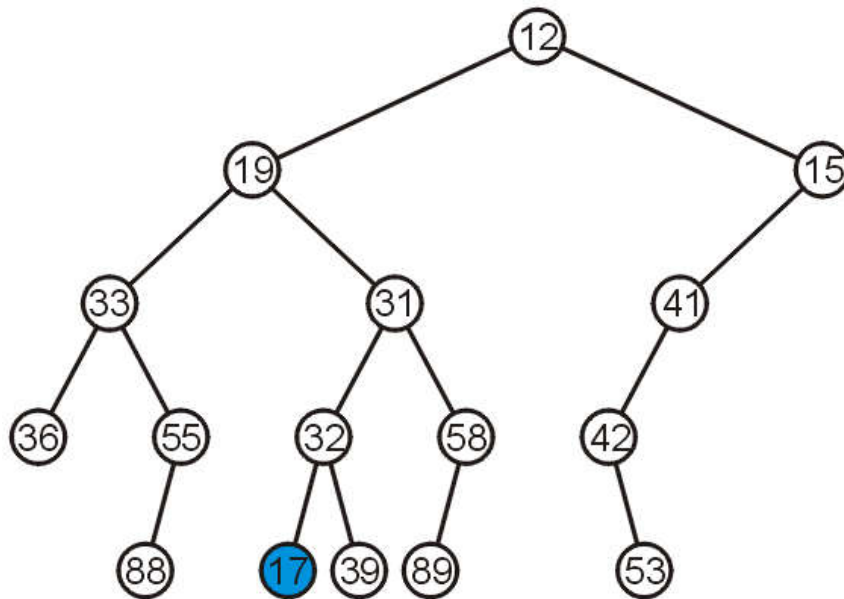
Inserting into a binary heap is done:

- At a leaf (move it up if it is smaller than the parent)

Push

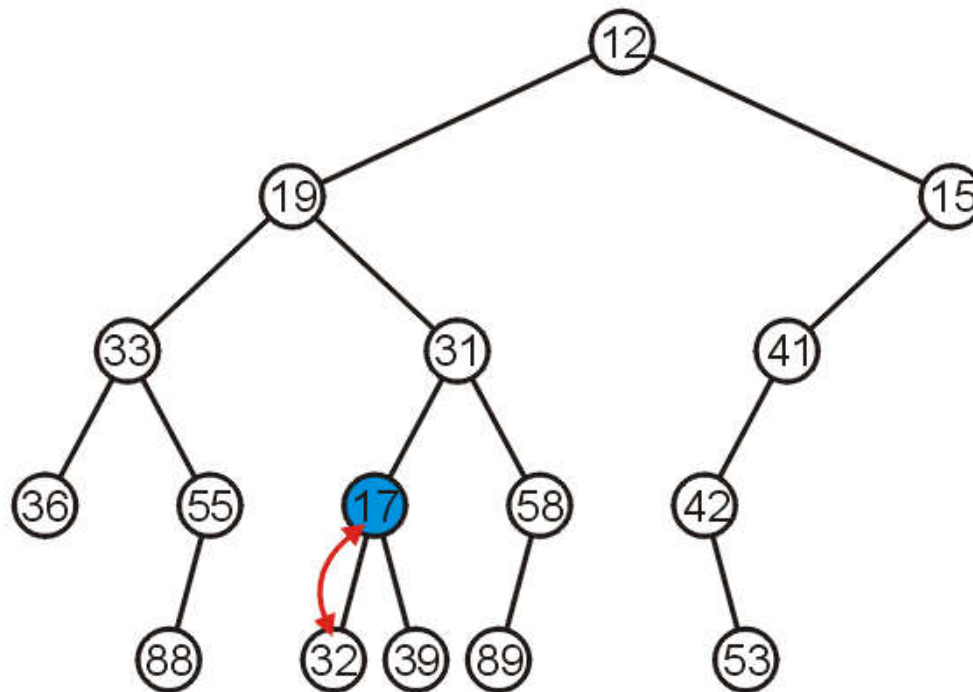
Inserting 17 into the last heap

- Select an **arbitrary** node to **insert a new leaf node**:



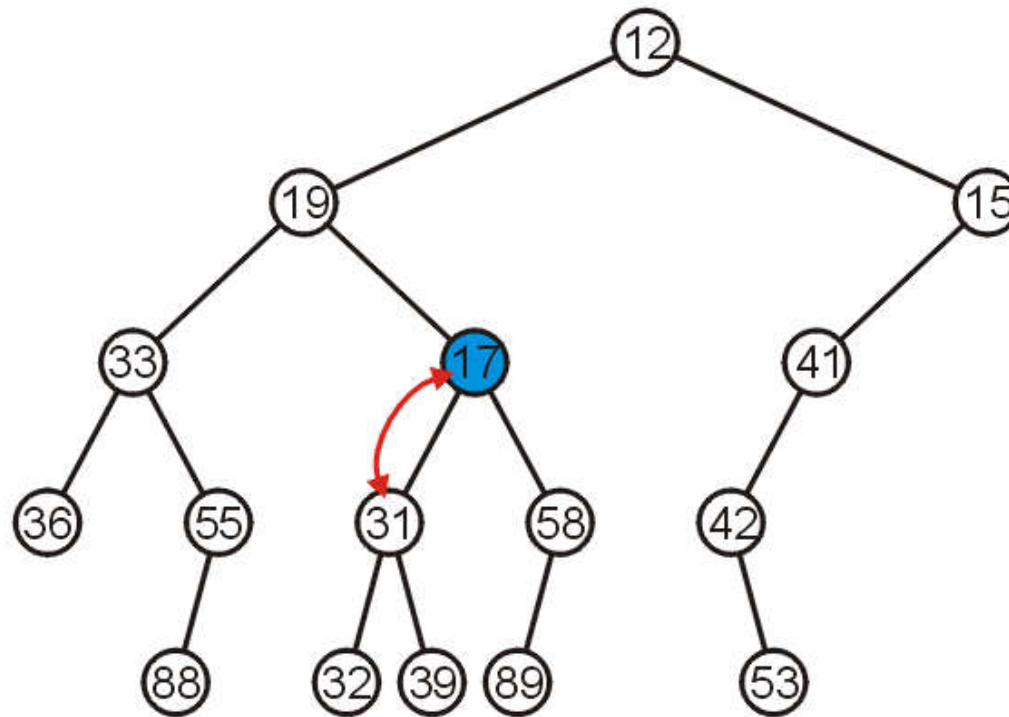
Push

The node 17 is less than the node 32, so we swap them



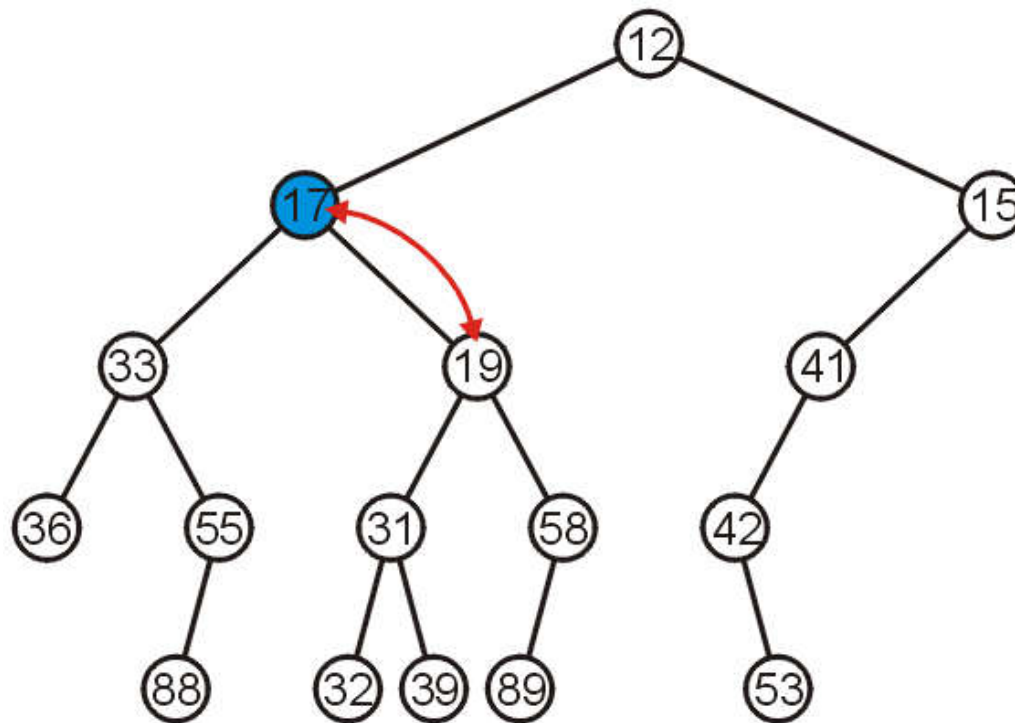
Push

The node 17 is less than the node 31; swap them



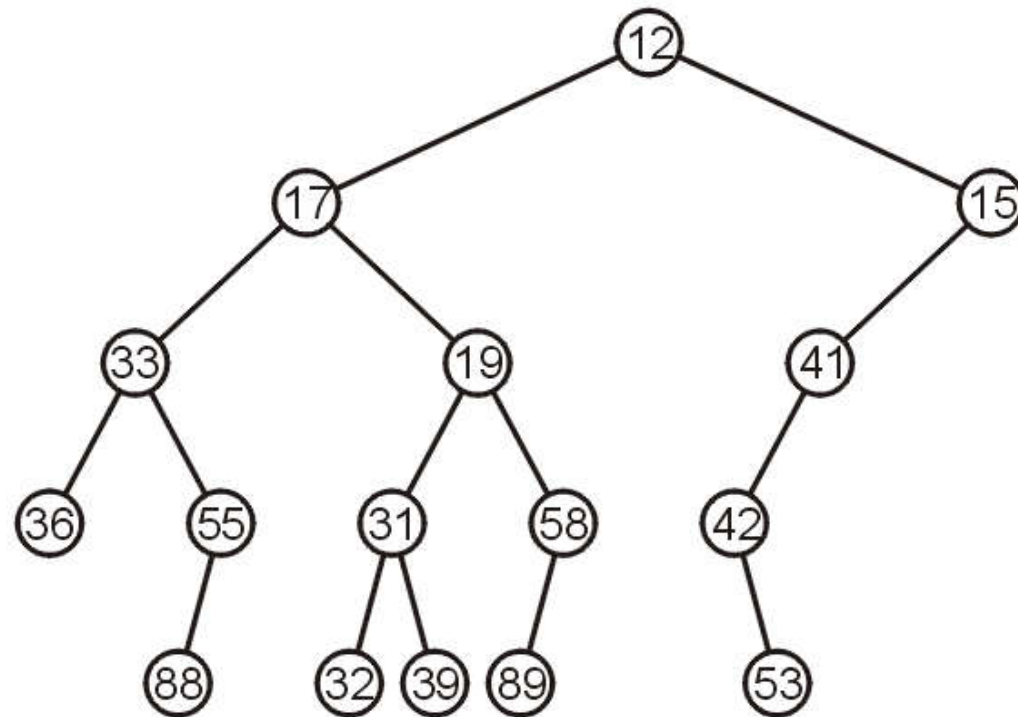
Push

The node 17 is less than the node 19; swap them



Push

The node **17** is **greater** than **12** so **we** are **finished**



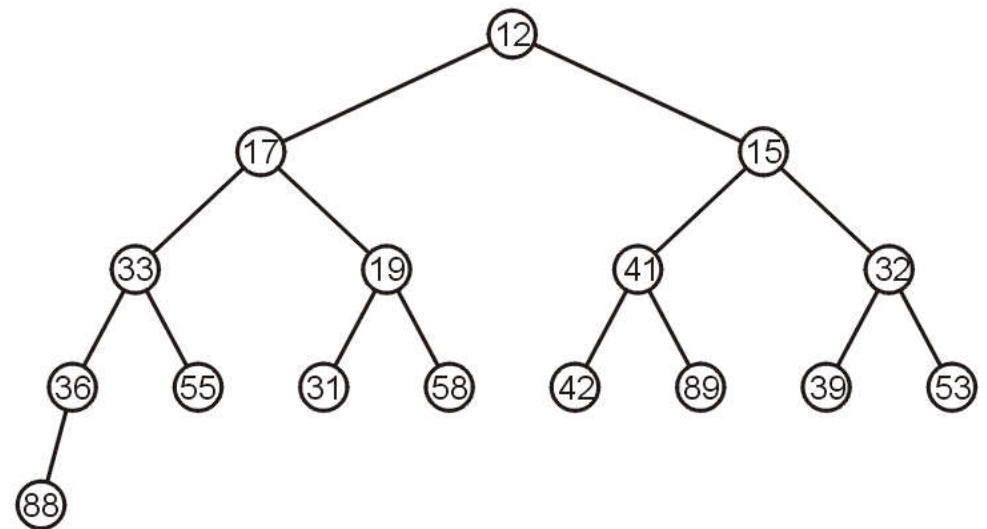
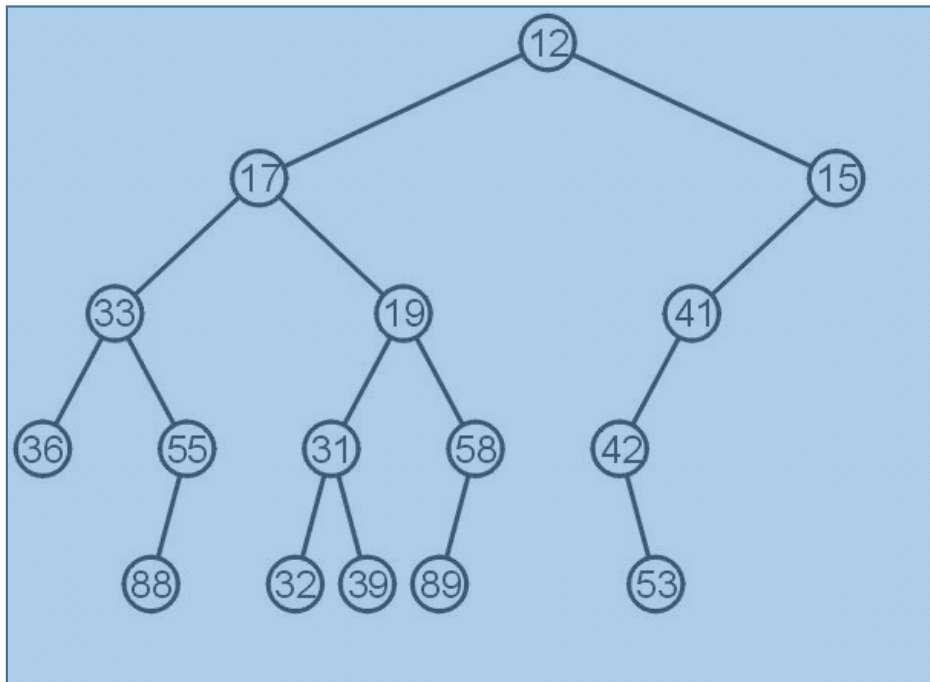
Push

Observation: both the **left** and **right** subtrees of 19 **were greater than 19**, thus **we are guaranteed** that we don't have to send the new node down

This process is called *percolation*, that is, the **lighter** (smaller) **objects move up** from the bottom of the min-heap

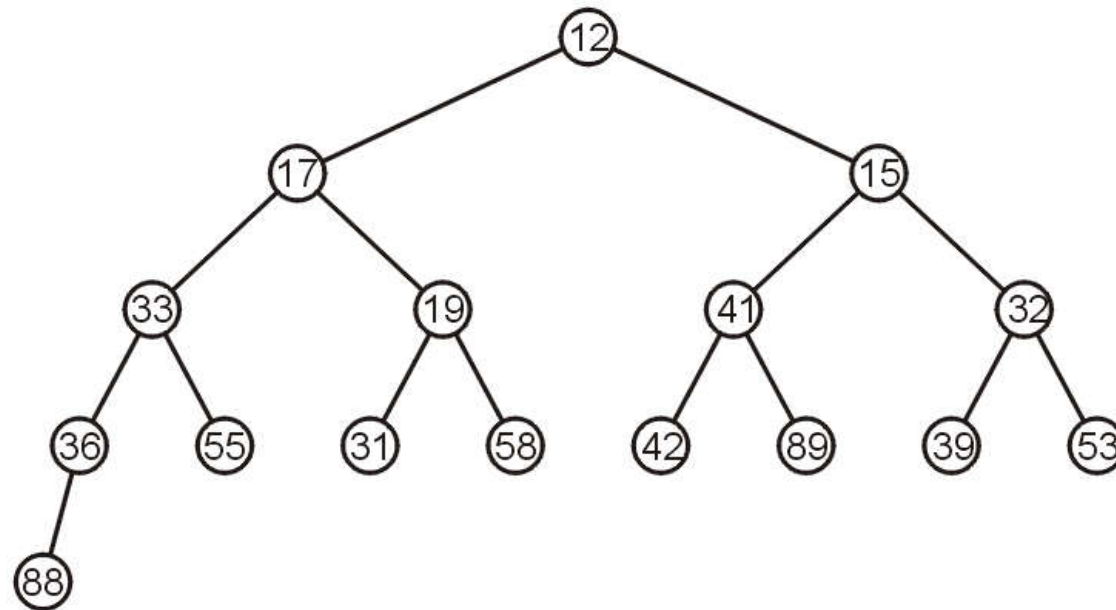
Implementing Heap as a Complete Binary Tree

For example, the **previous heap** may be **represented** as the following (**non-unique!**) **complete binary tree**:



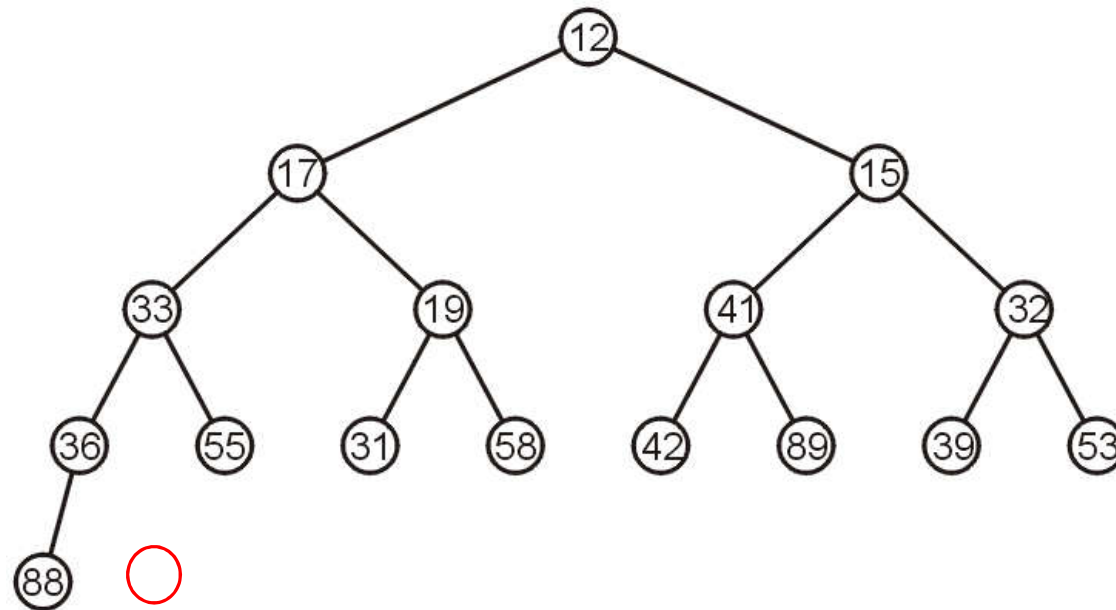
Complete Binary Trees: Push

If we **insert** into a complete Binary tree, we need only **place** the new node **as a leaf node** in the **appropriate location** and percolate up



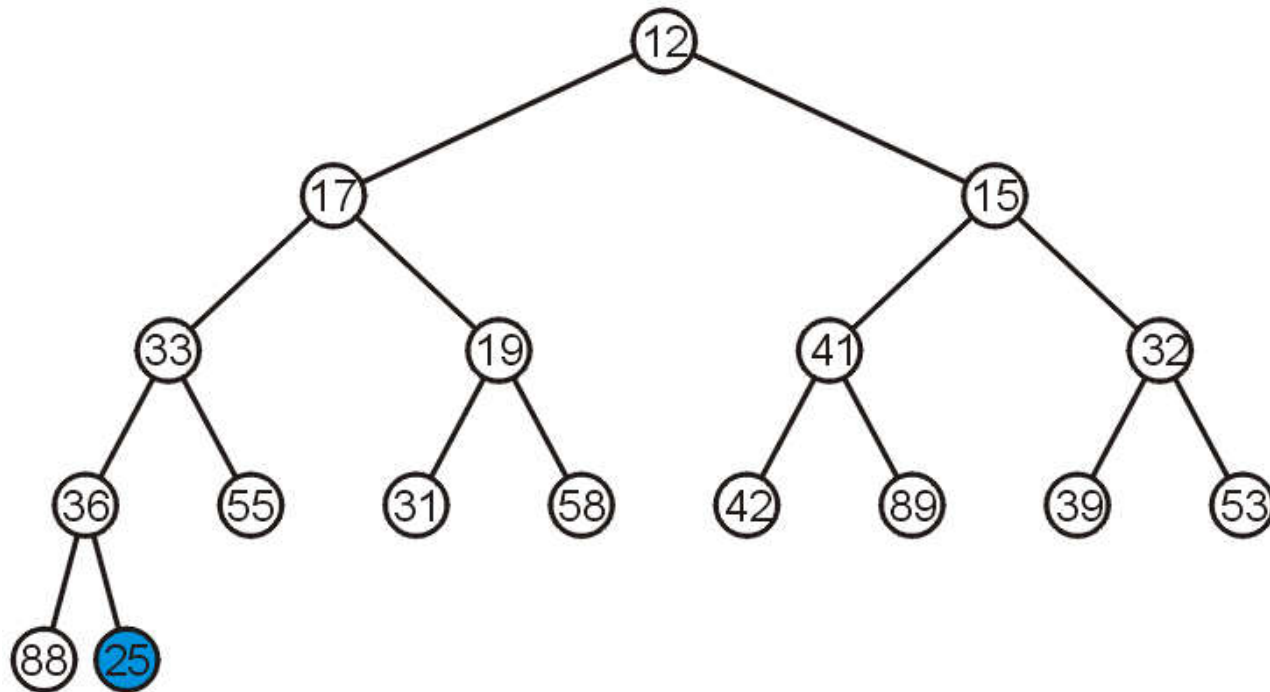
Complete Binary Trees: Push

If we insert into a complete Binary tree, we need only place the new node as a leaf node in the **appropriate location** and percolate up



Complete Binary Trees: Push

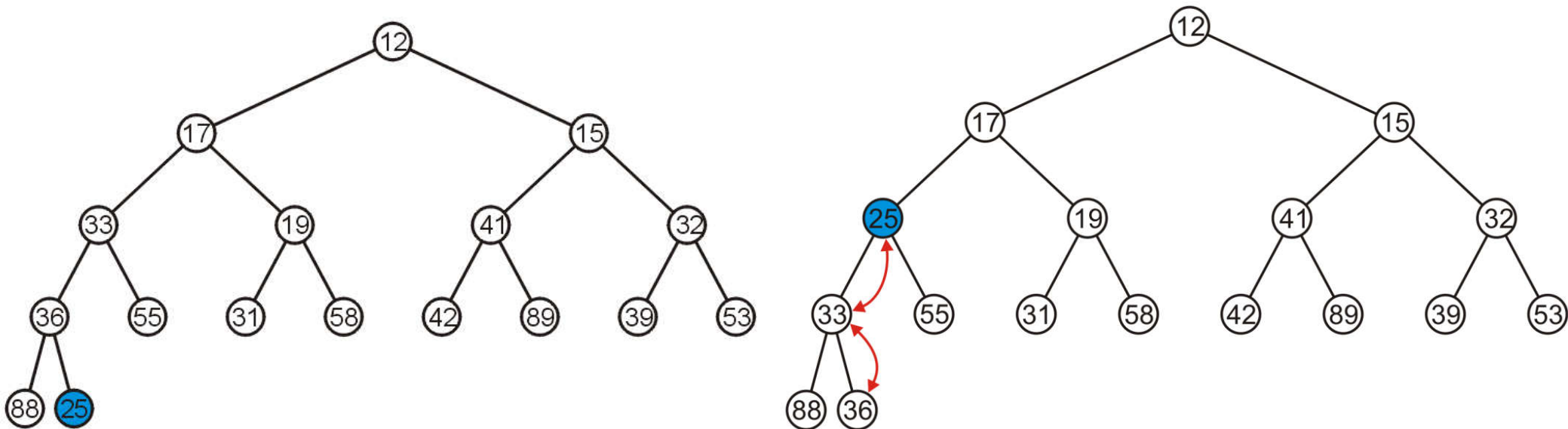
For example, push 25:



Complete Trees: Push

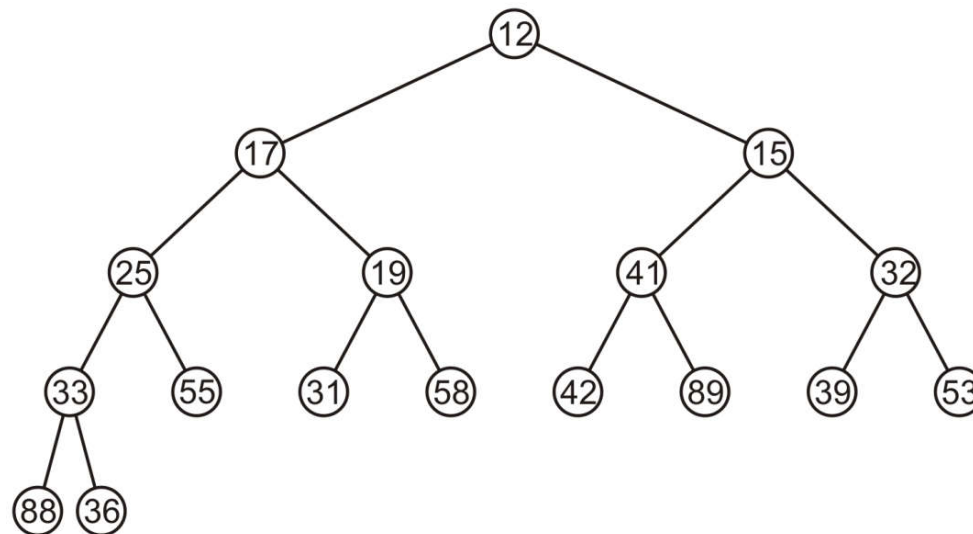
We have to **percolate 25 up** into its appropriate location

- The resulting heap is still a complete tree



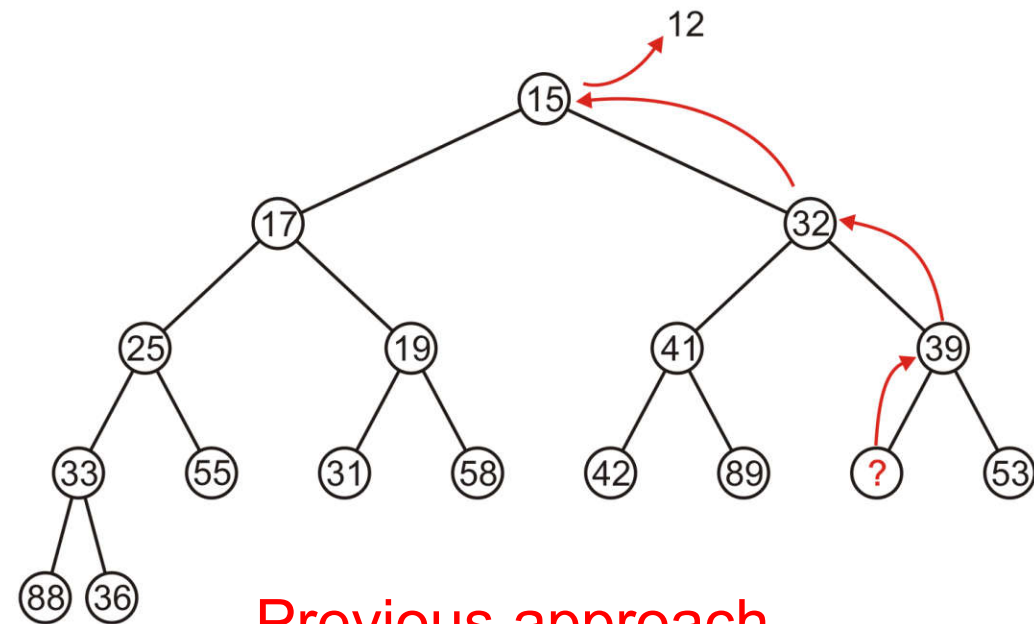
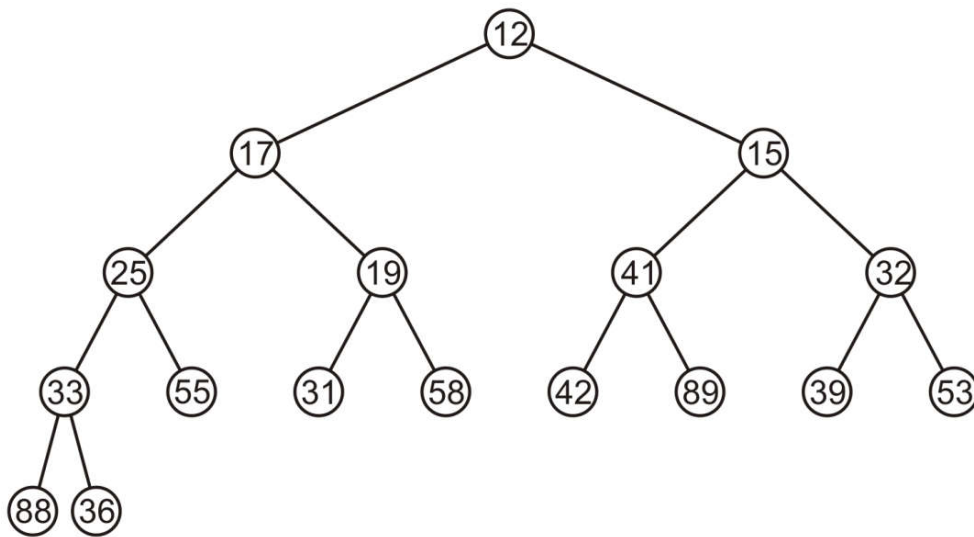
Complete Trees: Pop

Suppose we want to pop the top entry: 12



Complete Trees: Pop

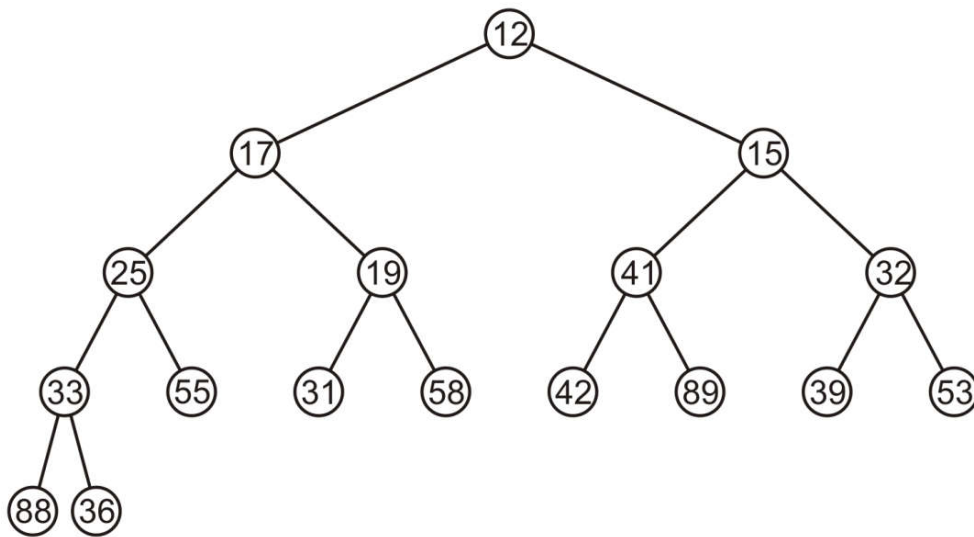
Percolating up **creates a hole** leading to a **non-complete tree**



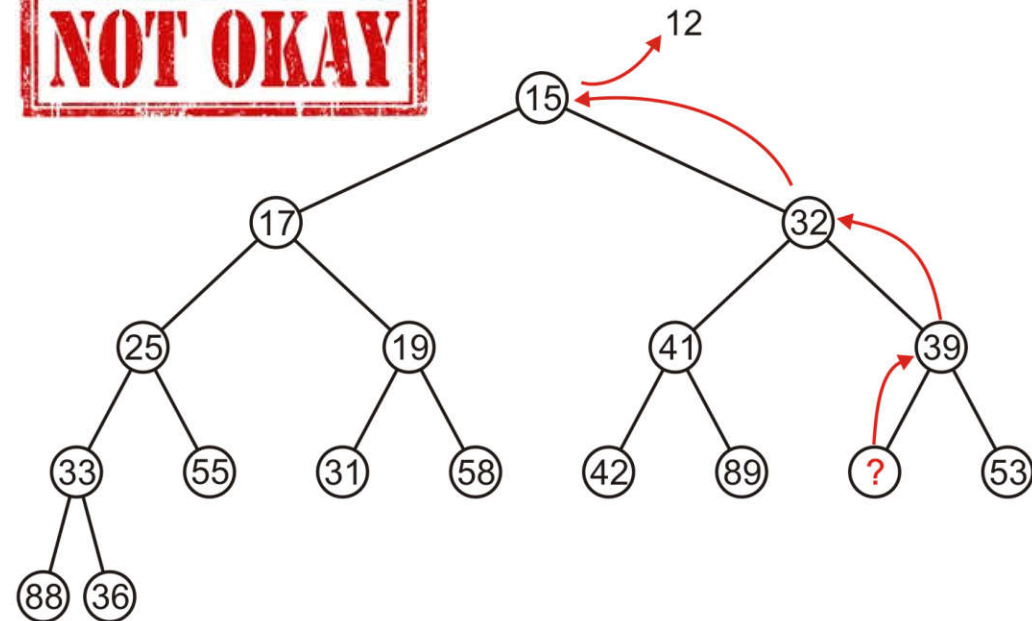
Previous approach

Complete Trees: Pop

Percolating up **creates a hole** leading to a **non-complete tree**

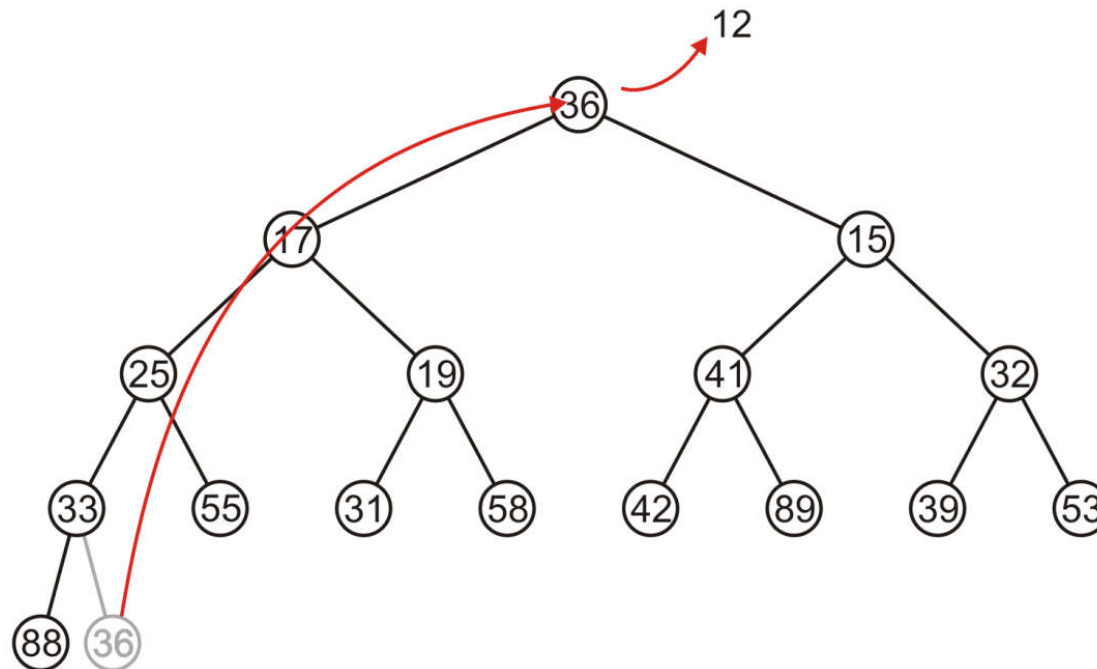


NOT OKAY



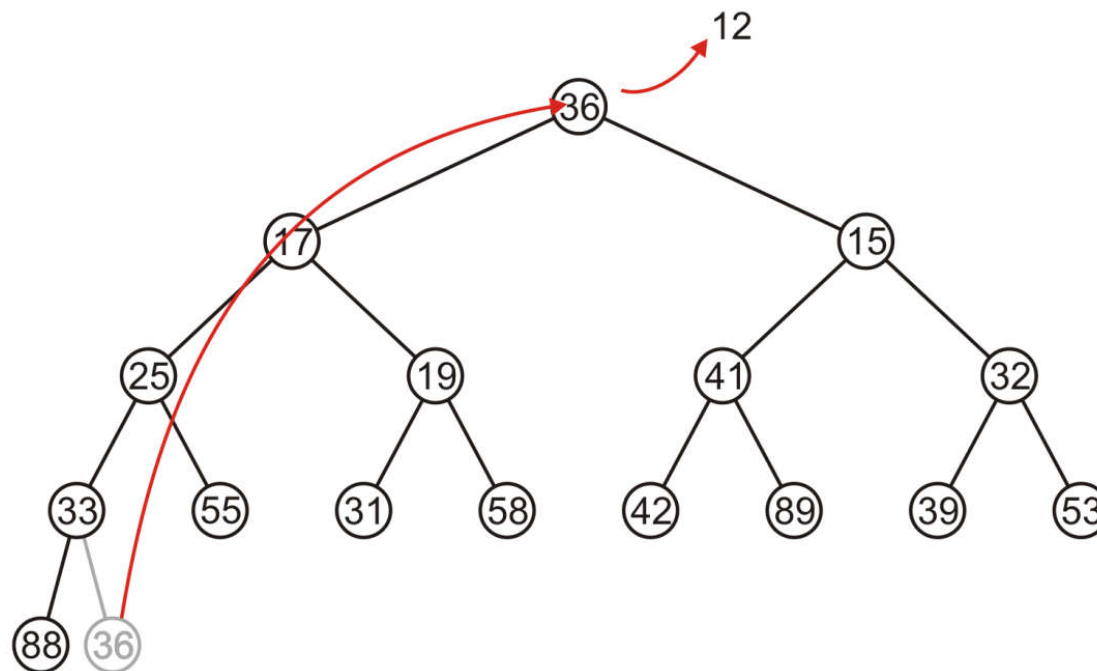
Complete Trees: Pop

Alternatively, **copy the last entry** in the heap **to the root**



Complete Trees: Pop

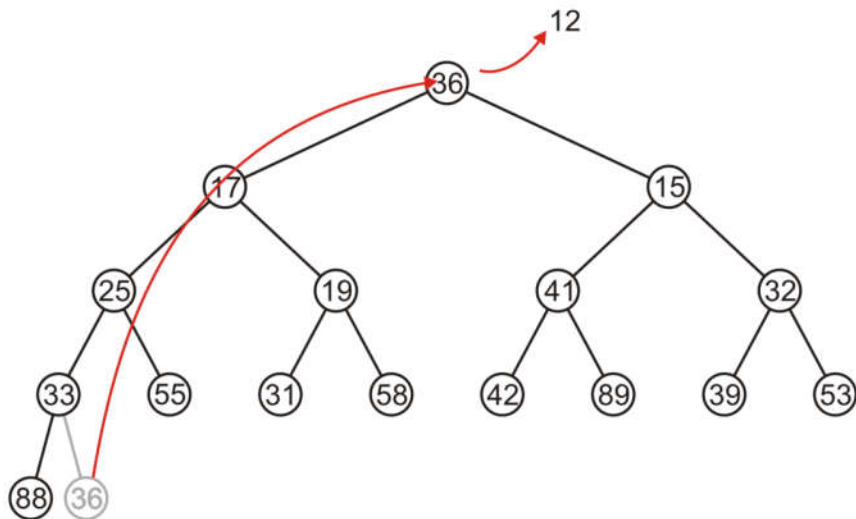
Now this is NOT a min-heap



Complete Trees: Pop

Now, **percolate 36 down** swapping it with the smallest of its children

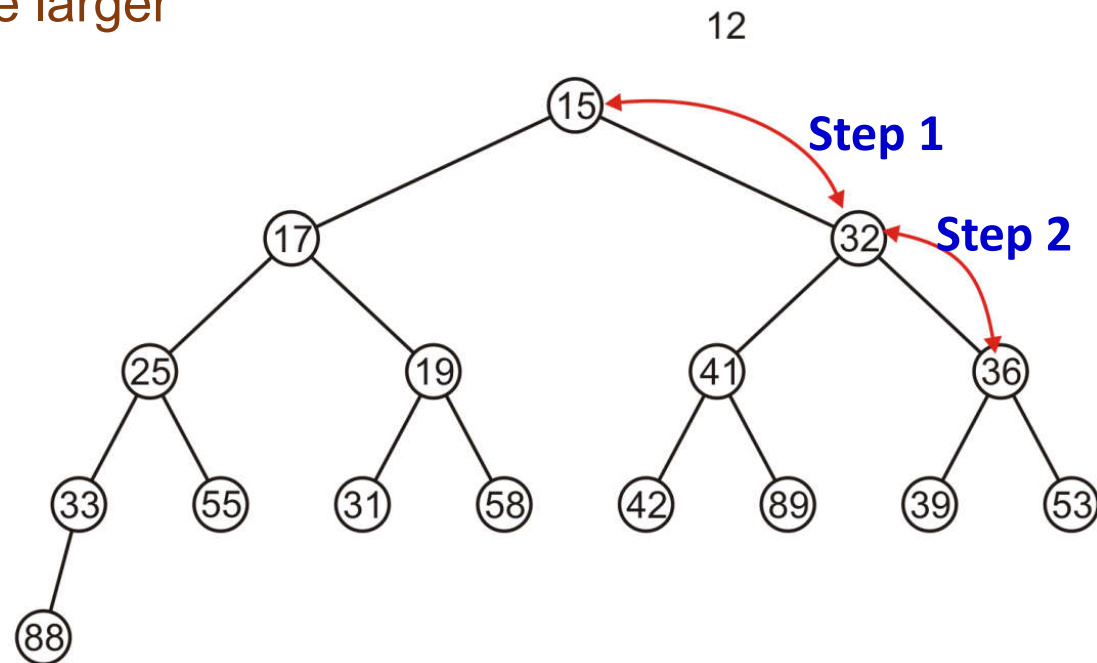
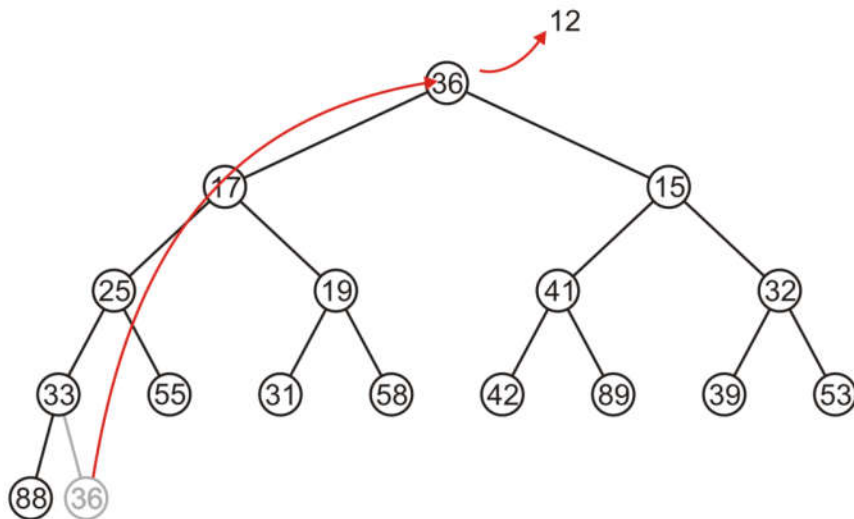
- We **halt** when both **children are larger**



Complete Trees: Pop

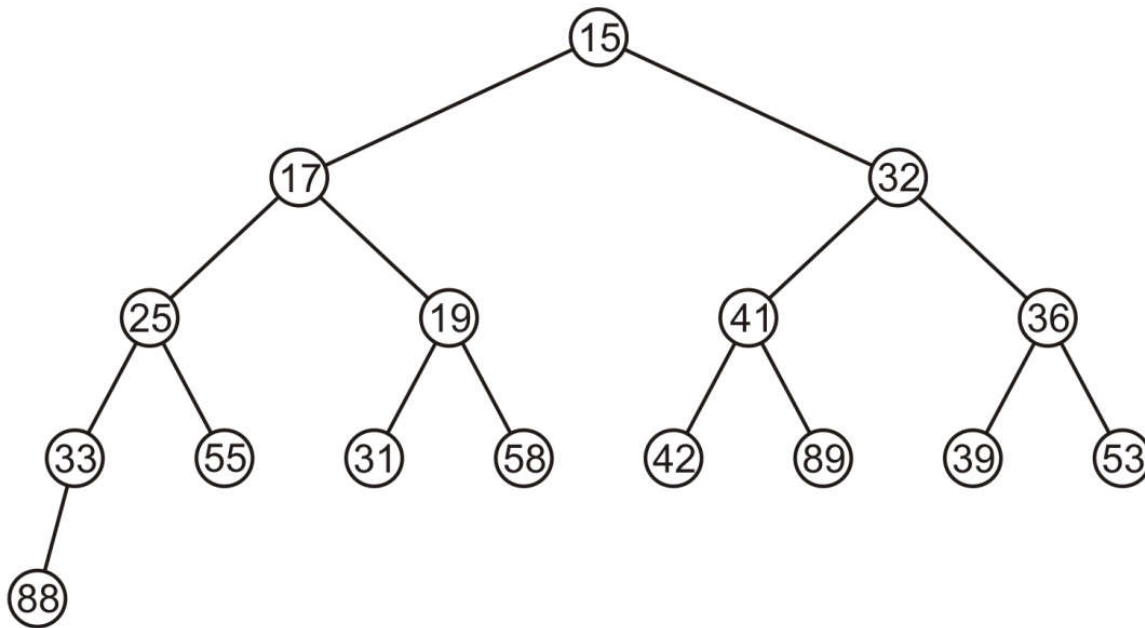
Now, **percolate 36 down** swapping it with the smallest of its children

- We **halt** when both children are larger



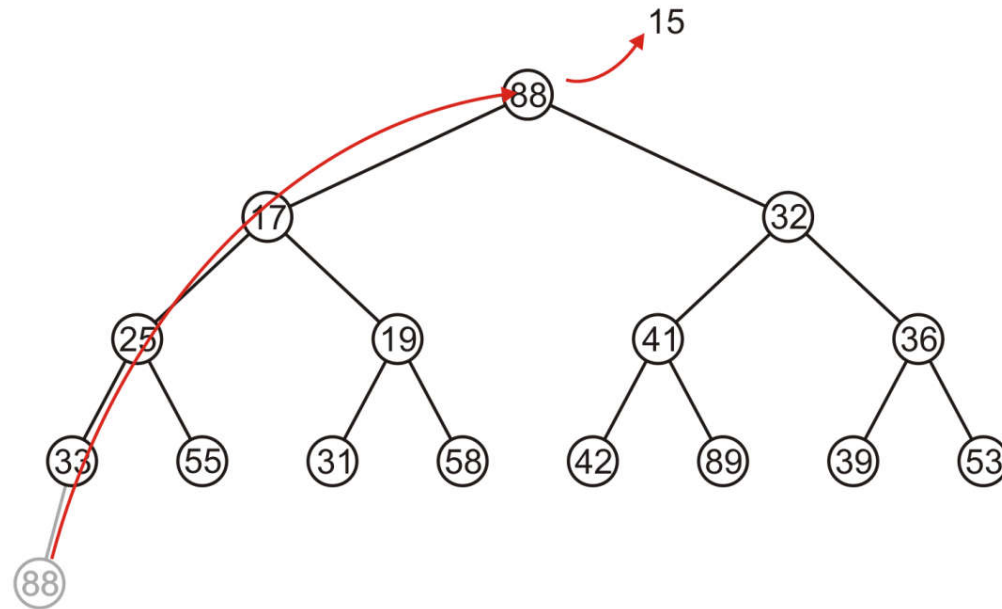
Complete Trees: Pop

The resulting tree is now still a complete tree:



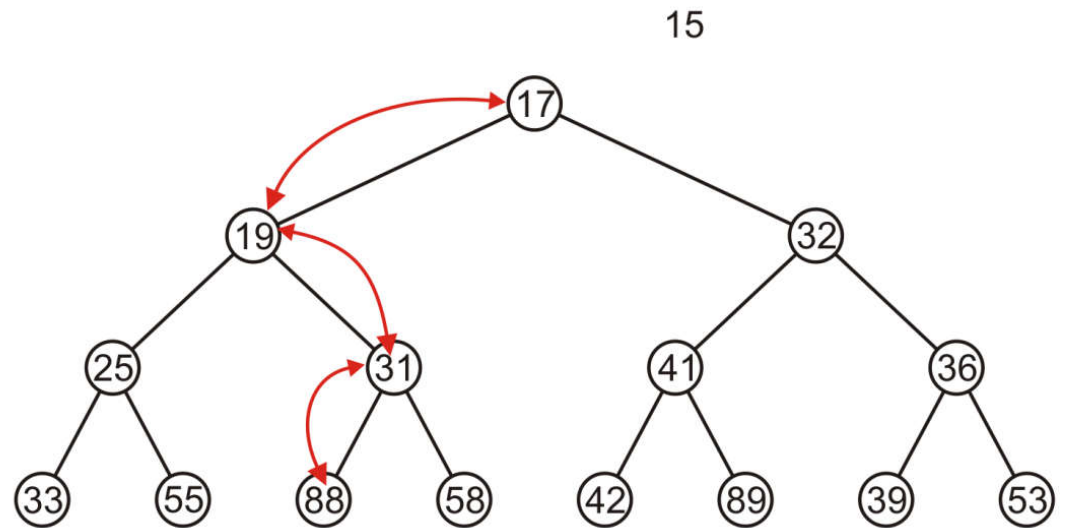
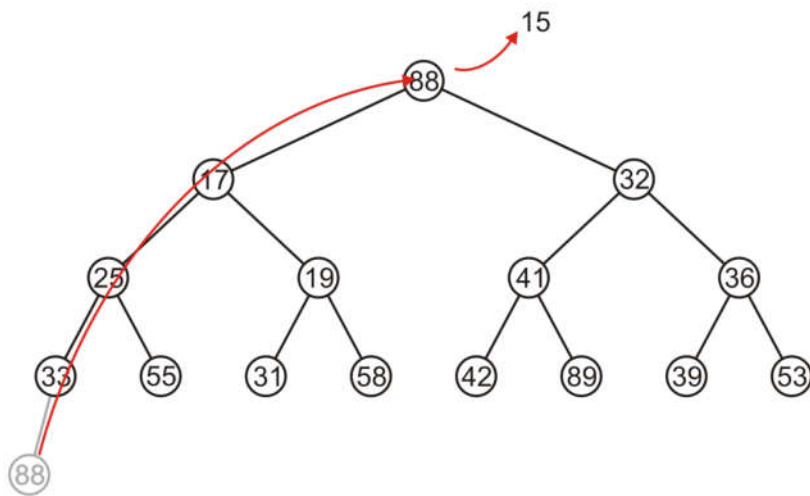
Complete Trees: Pop

Again, popping 15, copy up the last entry: 88



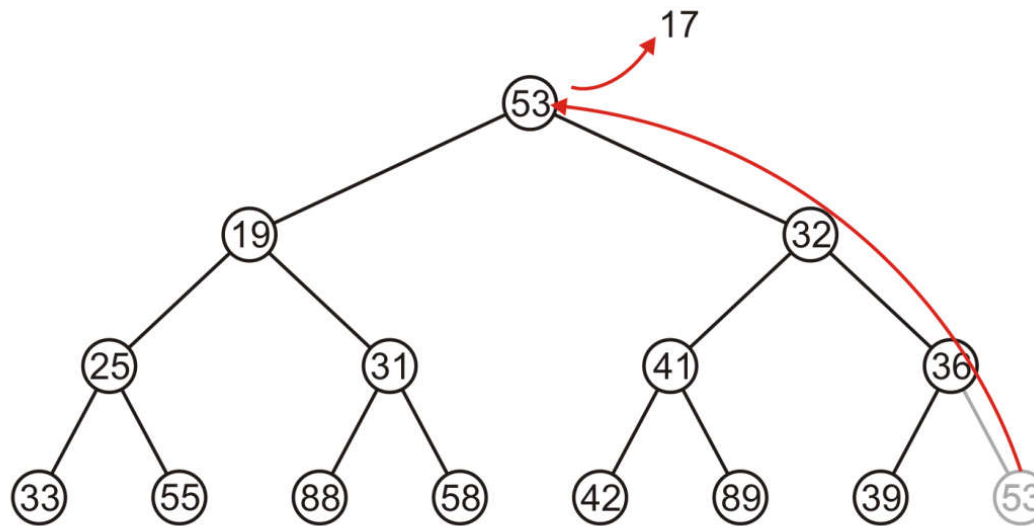
Complete Trees: Pop

This time, it gets percolated down to the point where it has no children



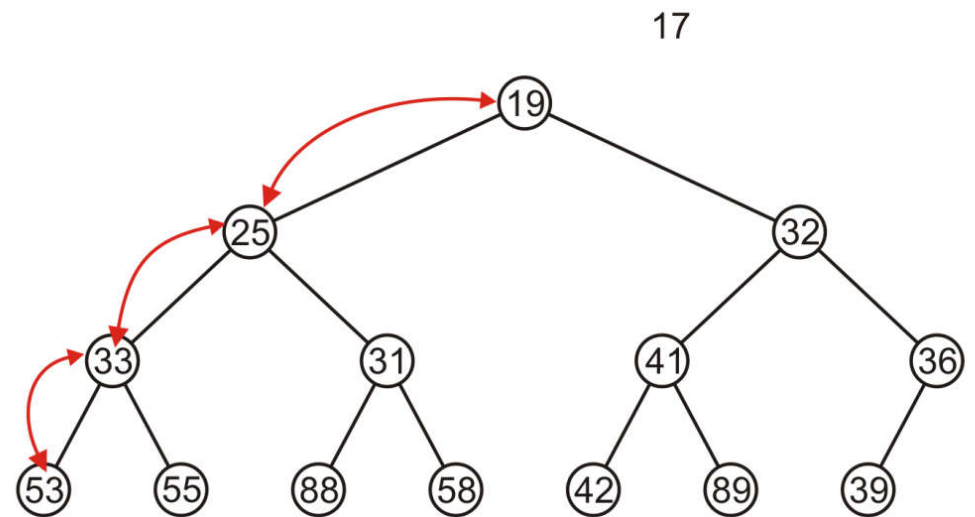
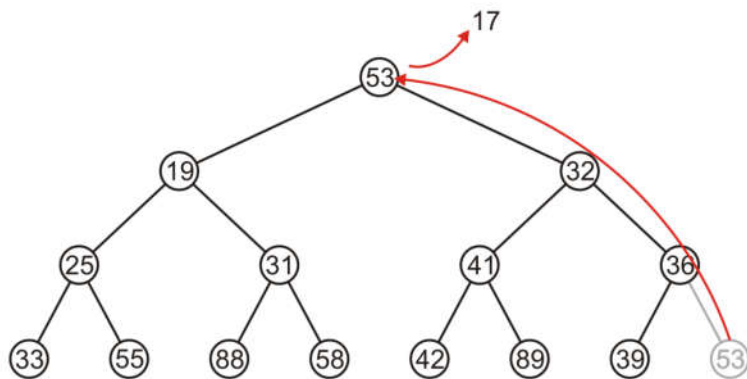
Complete Trees: Pop

In popping 17, 53 is moved to the top



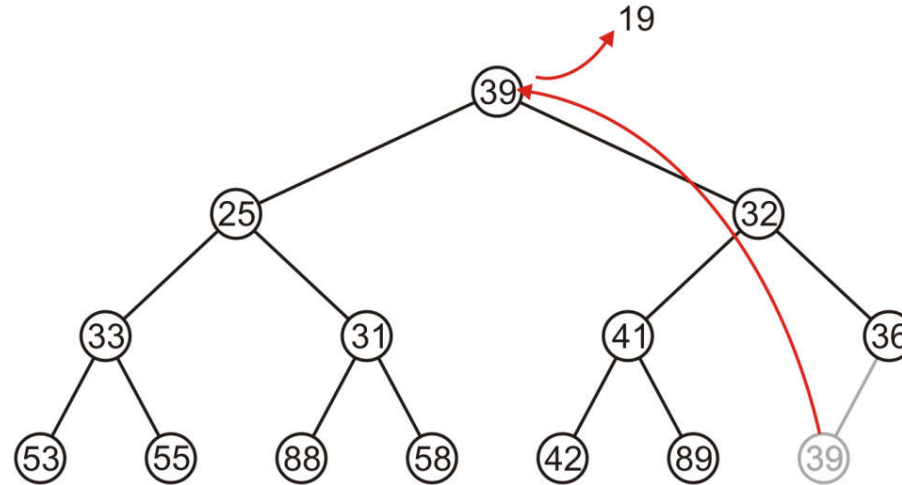
Complete Trees: Pop

And **percolated down**, again to the deepest level



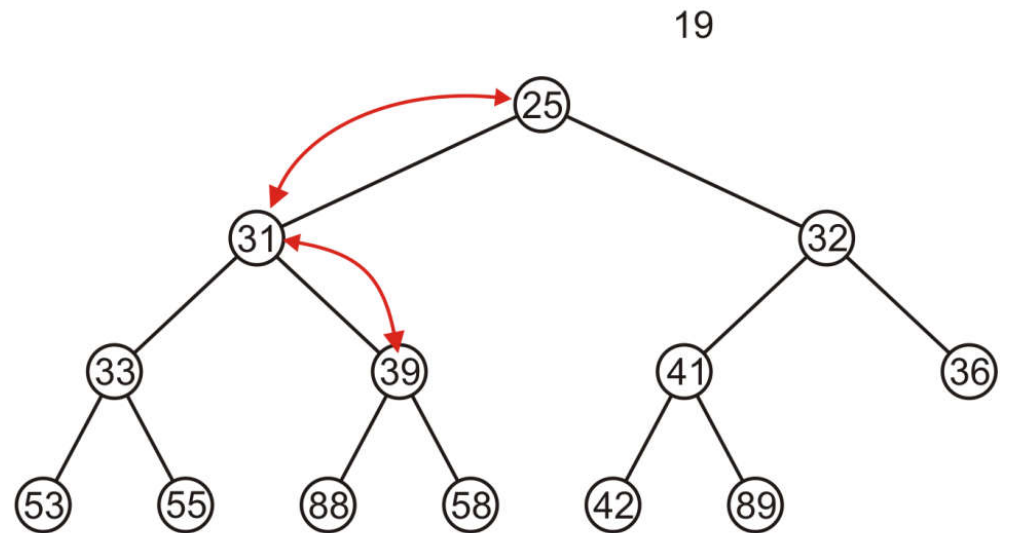
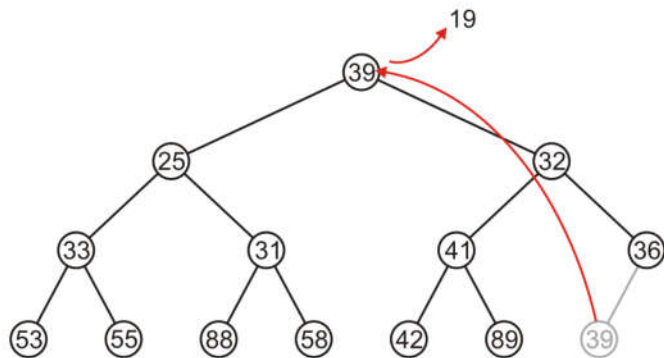
Complete Trees: Pop

Popping 19 copies up 39



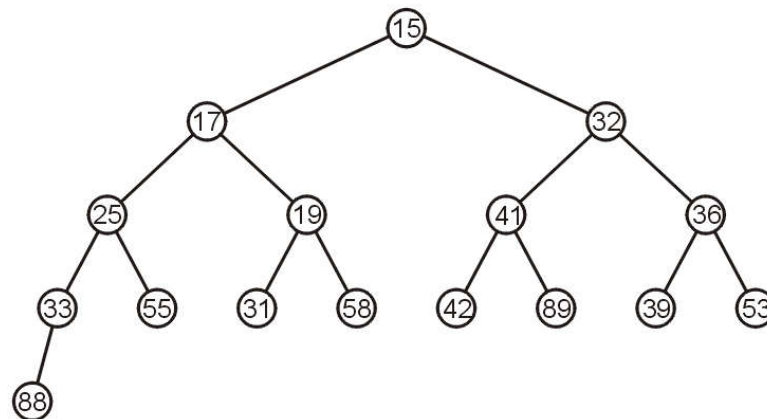
Complete Trees: Pop

Which is then **percolated down** to the **second deepest level**



Array Implementation

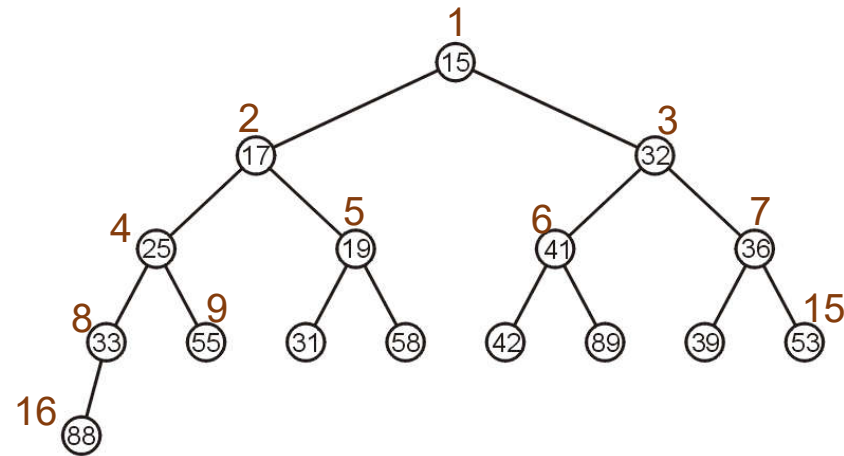
For the heap



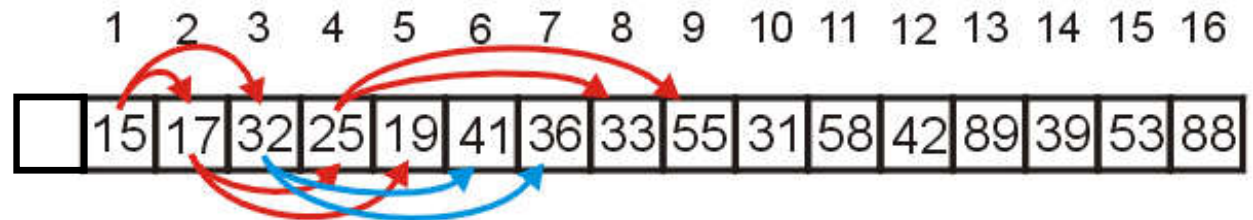
a level order traversal yields:

	15	17	32	25	19	41	36	33	55	31	58	42	89	39	53	88
--	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

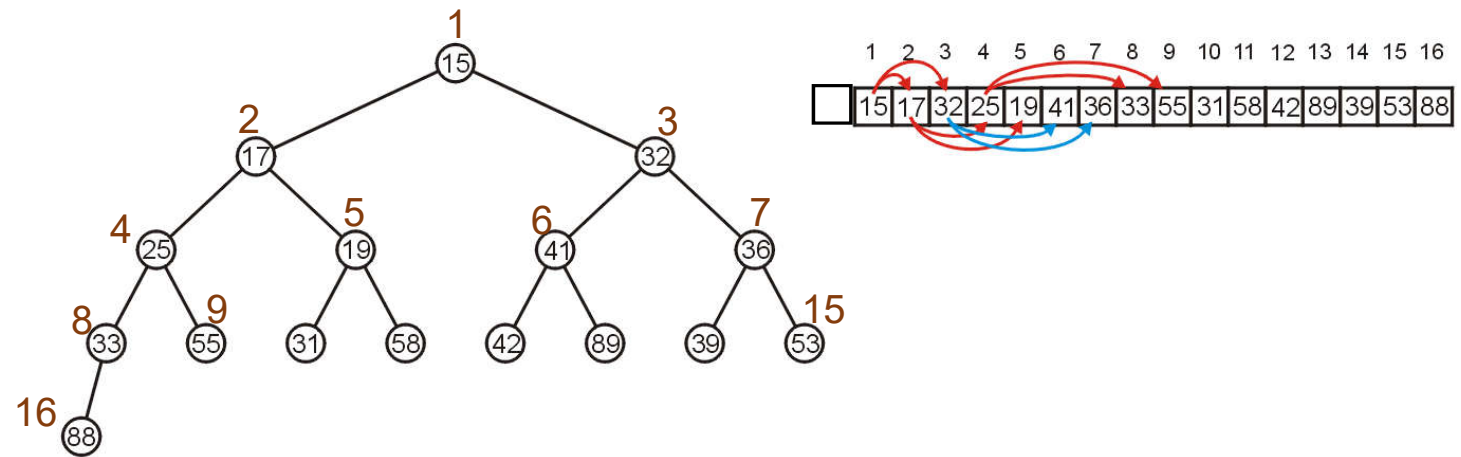
Array Implementation



Recall that If we associate an index—starting at 1—with each entry in the breadth-first traversal, we get:



Array Implementation



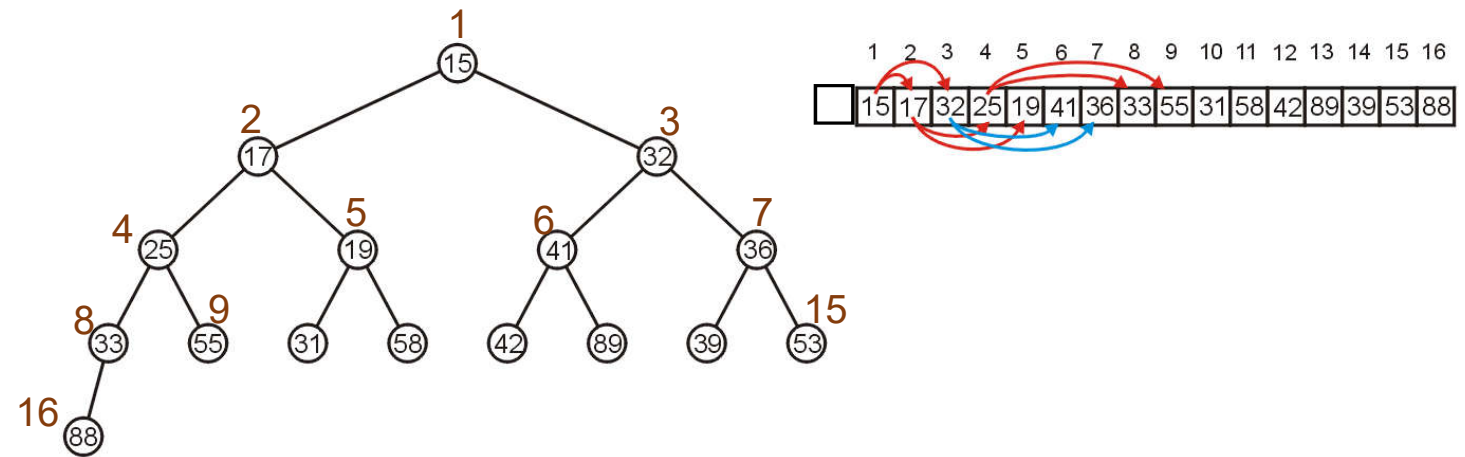
Given the entry at index k , it follows that:

- The parent of node is a $k/2$
- the children are at $2k$ and $2k + 1$

```
parent = k >> 1;  
left_child = k << 1;  
right_child = left_child + 1;
```

Cost (trivial): start array at position 1 instead of position 0

Array Implementation



Min-heap property to be maintained:

$$Array[parent(i)] \leq Array[i]$$